



CPT316: Programming Language Implementation and Paradigms
SCHOOL OF COMPUTER SCIENCES UNIVERSITI SAINS MALAYSIA

Semester I,

Academic Session: 2025/2026

Group Number : 24

Assignment 1

Ts. Dr. Ramona Binti Ramli

Name	Matric No	USM Email Address
AHMED M.H. TAHA	22304588	ahmedtaha@student.usm.my
ZHOU YIANG	22305072	evanzhou@student.usm.my
ABDULLAH ABDULRAHMAN SALEM OMAR AL-BAKER	2230585	abdullahalbaker@student.usm.my
AL Rasyidiqy Muhammad Khathir	165293	khathir@student.usm.my

Table of Contents

1. Programming Paradigm and Justification	3
2. Design and Implementation	4
2.1 Lexical Analyzer Design	4
2.2 Syntax Analyzer Design.....	6
2.3 Error Handling	7
3. Demonstration Evidence	7
3.1 Valid Input Test Cases	7
3.2 Error Test Cases.....	9
4. Reflection	10
5. Reference	10

1. Programming Paradigm and Justification

For this mini compiler, we decided to use the Object-Oriented Programming (OOP) paradigm with C++. We chose OOP mainly because our implementation naturally splits into different components lexer, parser, syntax tree nodes and each part need to maintain its own internal state. Using separate classes made the project more manageable and easier for us to organize.

The process of building a compiler fits well with object-oriented design principles. A compiler has different components that work together in a sequence. The lexical analyzer reads source code and produces tokens. These tokens then go to the syntax analyzer, which builds parse trees. This separation of tasks matches OOP's focus on modularity and encapsulation.

In our implementation, we created four main classes, each with a specific role. The Token class represents individual lexical units along with their type, value, and position information. The Lexer class handles the entire process of tokenization, including managing whitespace, classifying characters, and detecting invalid tokens. The Parser class uses recursive descent parsing to check for syntax errors and build the syntax tree. Finally, the SyntaxTreeNode class models tree structures with parent-child relationships.

Compared to functional or procedural approaches, this class-based organization has a number of advantages. It first establishes distinct lines between the various compilation stages. The internal workings of the parser are not necessary for the lexer to understand, and vice versa. Second, because each class can be changed or debugged separately, it improves code maintainability. Thirdly, it allows for extensibility, meaning that we could add new token types or grammar rules without having to completely rewrite the program.

Complex state management is another area in which OOP shines. The lexer must keep track of where it is in the input string, count the various token types, and maintain arrays of valid and invalid tokens. The parser needs to be able to identify errors and remember where it is in the token stream. This state is kept organized and unintentional changes from outside code are avoided by using classes with private member variables.

The procedural approach requires managing state through global variables or complex parameter lists and passing multiple variables between functions when comparing OOP and procedural programming for this task. As the compiler gets bigger, this quickly gets messy. The stateful nature of lexical analysis and parsing, where you must maintain position, accumulate tokens, and handle errors, makes functional programming difficult to use, even though it can be elegant for certain tasks.

C++ was specifically chosen because of its ability to work with performance characteristics. C++ offers low-level control, effective memory management, and the object-oriented features we require. By implementing dynamic arrays manually using the language, we can precisely control memory allocation and avoid relying on contemporary libraries that might not compile in older development environments.

2. Design and Implementation

2.1 Lexical Analyzer Design

The goal of the lexical analyzer, implemented by the Lexer class, is to convert raw input strings into token sequences. This process mainly comprises two steps: tokenization to identify and classify meaningful units, and scanning that can handle basic tasks such as removing whitespace.

Our lexer processes input character by character, from left to right. At each position, it looks at the current character and makes a decision about the type of token it belongs to. Its recognition logic follows these rules:

For identifiers, we check that the character is a lowercase letter between 'a' and 'z'. Since identifiers must be single lowercase letters, we accept exactly one character and move on. For numbers, we detect if the current character is a digit, then continue gathering consecutive digits to form a complete integer. This implements token pattern matching where we build up the lexeme by consuming characters while they match the expected pattern.

Operators (+, -, *, /) are single-character tokens. If we see one of these characters, we can immediately decide what it is and move on. Similarly for the assignment operator (=), for parentheses, and for the semicolon statement terminator. Anything else is an error; we note its position and value for error reporting.

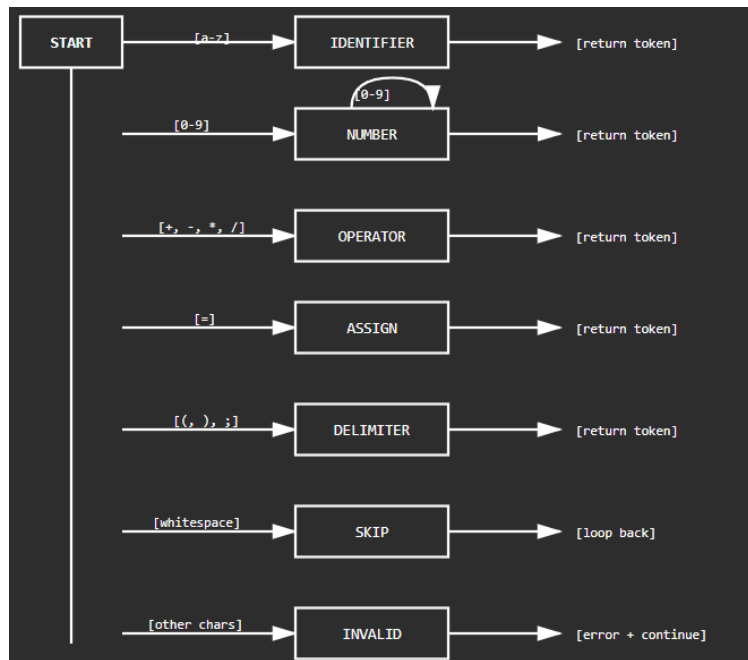


Figure 1: shows the **State Diagram for Lexical Analyzer**

The lexical analysis process can be described with the following pseudocode:

```
Function tokenize(input):  
    position = 0  
    tokens = empty array  
  
    While position < length(input):  
        skip whitespace characters  
        current = input[position]  
  
        If current is lowercase letter [a-z]:  
            create IDENTIFIER token  
            position = position + 1  
  
        Else if current is digit [0-9]:  
            number = ""  
            While current is digit:  
                number = number + current  
                position = position + 1  
            create NUMBER token with value = number  
  
        Else if current in {+, -, *, /}:  
            create OPERATOR token  
            position = position + 1  
  
        Else if current is '=', '(', ')', or ';':  
            create appropriate token type  
            position = position + 1  
  
        Else:  
            create INVALID token  
            report lexical error  
            position = position + 1  
  
    Return tokens
```

The language defines a limited character set. Anything outside this set represents a lexical error. Our implementation handles these errors gracefully by creating INVALID tokens that store the problematic character and its position. When invalid tokens are encountered, we continue scanning rather than immediately terminating, allowing us to report all lexical errors in a single pass. The invalid tokens are displayed separately with messages in the format: "LexicalError at position X: Invalid character 'Y'".

The lexer maintains separate counters for different token categories: identifiers, numbers, operators, and other tokens (parentheses, semicolons, assignment). These counters increment as tokens are recognized. At the end of tokenization, we can report both the total token count and the breakdown by type, giving users insight into the composition of their input.

2.2 Syntax Analyzer Design

The syntax analyzer, or parser, validates that token sequences conform to the grammar. Our implementation uses recursive descent parsing, a top-down parsing technique that directly mirrors the grammar structure.

The language grammar is defined as:

$\langle \text{stmt} \rangle \rightarrow \text{id} = \langle \text{expr} \rangle ;$

$\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle \mid \langle \text{expr} \rangle '+' \langle \text{term} \rangle \mid \langle \text{expr} \rangle '-' \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{factor} \rangle \mid \langle \text{term} \rangle '*' \langle \text{factor} \rangle \mid \langle \text{term} \rangle '/' \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow \text{id} \mid \text{int} \mid (\langle \text{expr} \rangle)$

This grammar exhibits left recursion in the $\langle \text{expr} \rangle$ and $\langle \text{term} \rangle$ productions. Left recursion occurs when a non-terminal appears as the leftmost symbol in its own production rule, like $\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle '+' \langle \text{term} \rangle$. This creates a problem for recursive descent parsers because the parsing function would call itself immediately before consuming any input, leading to infinite recursion.

To solve this, we apply the left recursion elimination transformation rule:

Original: $A \rightarrow A\alpha \mid \beta$

Transformed: $A \rightarrow \beta A'$ and $A' \rightarrow \alpha A' \mid \epsilon$

Applying this transformation converts the left-recursive rules into iterative structures. Our implementation handles this by converting the recursive structure into an iterative loop. For expressions, we first parse a term, then loop to collect additional terms separated by + or - operators. The same transformation applies to terms with * and / operators.

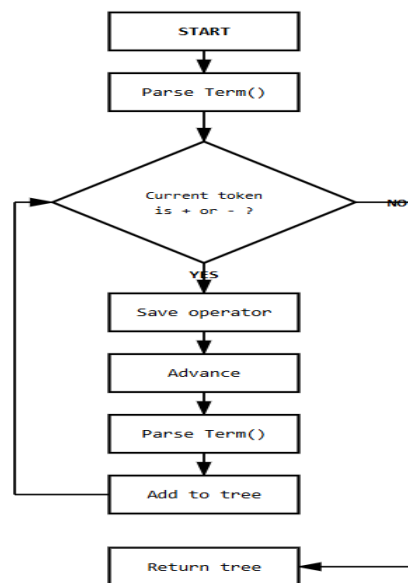


Figure 2: Shows the Flowchart for Parsing an Expression in the Recursive Descent Parser.

As the parser validates the input, it simultaneously builds a syntax tree representation. The `SyntaxTreeNode` class creates nodes labeled with grammar symbols or token values. Child nodes are connected to parent nodes to form the hierarchical structure. For example, parsing "`x = 3 + 5;`" produces a tree with `<stmt>` as the root. The root has four children: the identifier '`x`', the assignment operator '`=`', an `<expr>` subtree, and the semicolon '`;`'. The `<expr>` subtree contains `<term>` nodes and operators, breaking down the structure according to the grammar's operator precedence rules. The tree visualization uses a text-based diagram with connecting lines to give users a clear visual representation of how their input was parsed.

2.3 Error Handling

Error detection and reporting is crucial for making the compiler useful. Our implementation distinguishes between lexical errors and syntax errors, providing specific messages for each.

Lexical errors occur when the scanner encounters characters that don't belong to any valid token type. The error message reports the exact position: "LexicalError at position X: Invalid character 'Y'". The lexer continues scanning after finding invalid characters to catch all errors in one pass. However, if any invalid tokens exist, the parser skips syntax analysis entirely since syntax checking requires valid tokens.

Syntax errors occur when valid tokens appear in incorrect sequences. Our parser detects missing components ("Missing ';'"), unexpected tokens ("Expected 'Y' but found 'Z'"), and premature end of input ("Unexpected end of input in expression"). All error messages follow a consistent format starting with the error type, including position information, and describing what went wrong in clear language.

3. Demonstration Evidence

3.1 Valid Input Test Cases

Test Case 1: Simple Assignment - `x = 5;`

Tokens: IDENTIFIER '`x`', ASSIGN '`=`', NUMBER '`5`', SEMICOLON '`;`'. Total 4 tokens (1 identifier, 1 number, 0 operators, 2 other). Parser builds syntax tree successfully with `<stmt>` as root containing identifier, assignment, expression with nested term and factor for number 5, and semicolon.

```
=====
INPUT: x=5;
=====
--- LEXICAL ANALYSIS ---
===== TOKEN STREAM =====
No.  Type      Value  Position
-----
1    IDENTIFIER x      0
2    ASSIGN    =      1
3    NUMBER    5      2
4    SEMICOLON ;      3
=====
===== TOKEN STATISTICS =====
Total tokens: 4
Identifiers: 1
Numbers: 1
Operators: 0
Other tokens: 2
=====
--- SYNTAX ANALYSIS ---
Parsing successful! Syntax tree generated.
===== SYNTAX TREE =====
+-- <stmt>
|  +-- x
|  +-- =
|  +-- <expr>
|      +-- <term>
|          +-- <factor>
|              +-- 5
|  +-- ;
=====
```

Test Case 2: Operator Precedence - $a = 2 + 3 * 4;$

Total 8 tokens (1 identifier, 3 numbers, 2 operators, 2 other). Tree demonstrates proper precedence with multiplication ($3 * 4$) nested deeper than addition, showing that $*$ binds tighter than $+$.

```
=====
INPUT: a = 2 + 3 * 4;
=====
--- LEXICAL ANALYSIS ---
===== TOKEN STREAM =====
No.  Type      Value      Position
-----
1    IDENTIFIER a          0
2    ASSIGN    =          2
3    NUMBER    2          4
4    PLUS      +          6
5    NUMBER    3          8
6    MULTIPLY  *         10
7    NUMBER    4         12
8    SEMICOLON ;         13
=====
===== TOKEN STATISTICS =====
Total tokens: 8
Identifiers: 1
Numbers: 3
Operators: 2
Other tokens: 2
=====
--- SYNTAX ANALYSIS ---
Parsing successful! Syntax tree generated.
===== SYNTAX TREE =====
+-- <stmt>
|   -- a
|   -- =
|   -- <expr>
|       -- <term>
|           -- <factor>
|               -- 2
|       -- +
|       -- <term>
|           -- <factor>
|               -- 3
|           -- *
|           -- <factor>
|               -- 4
|   -- ;
=====
```

Test Case 3: Parentheses - $z = (5 + 2) * 3;$

Total 10 tokens. Tree correctly shows parenthesized expression as single `<factor>` unit containing '(', `<expr>` for addition, and ')'. This factor then participates in multiplication.

```
=====
INPUT: z = (5 + 2) * 3;
=====
--- LEXICAL ANALYSIS ---
Valid Tokens:
Type      Value      Position
-----
IDENTIFIER z          0
ASSIGN    =          2
LPAREN    (          4
NUMBER    5          5
PLUS      +          7
NUMBER    2          9
RPAREN    )         10
MULTIPLY  *         12
NUMBER    3         14
SEMICOLON ;         15
=====
Token Statistics:
Identifiers: 1
Numbers: 3
Operators: 2
Other: 4
Total Valid: 10
Invalid: 0
=====
--- SYNTAX ANALYSIS ---
Parsing successful! Syntax tree generated.
===== SYNTAX TREE =====
+-- <stmt>
|   -- z
|   -- =
|   -- <expr>
|       -- <term>
|           -- <factor>
|               -- (
|                   -- <expr>
|                       -- <term>
|                           -- <factor>
|                               -- 5
|                       -- +
|                       -- <term>
|                           -- <factor>
|                               -- 2
|                   -- )
|           -- *
|           -- <factor>
|               -- 3
|   -- ;
=====
```


3.2 Error Test Cases

Lexical Error - $x = 5 \$ 3;$

Output: "LexicalError at position 6: Invalid character '\$". Parsing skipped due to lexical errors.

```
=====
INPUT: x = 5 $ 3;
=====

--- LEXICAL ANALYSIS ---

===== TOKEN STREAM =====
No.  Type      Value      Position
-----
1    IDENTIFIER x          0
2    ASSIGN    =          2
3    NUMBER    5          4
4    INVALID   $          6
5    NUMBER    3          8
6    SEMICOLON ;          9
=====

===== LEXICAL ERRORS =====
LexicalError at position 6: Invalid character '$'
=====

===== TOKEN STATISTICS =====
Total tokens: 6
Identifiers: 1
Numbers: 2
Operators: 0
Other tokens: 2
=====

--- SYNTAX ANALYSIS ---
Parsing skipped due to lexical errors.
```

Syntax Error: Unmatched Parenthesis - $a = (2 + 3;$

Output: "SyntaxError at position 10: Expected ')' but found ';'."

```
=====
INPUT: a = (2 + 3;
=====

--- LEXICAL ANALYSIS ---

===== TOKEN STREAM =====
No.  Type      Value      Position
-----
1    IDENTIFIER a          0
2    ASSIGN    =          2
3    LPAREN    (          4
4    NUMBER    2          5
5    PLUS      +          7
6    NUMBER    3          9
7    SEMICOLON ;         10
=====

===== TOKEN STATISTICS =====
Total tokens: 7
Identifiers: 1
Numbers: 2
Operators: 1
Other tokens: 3
=====

--- SYNTAX ANALYSIS ---
SyntaxError at position 10: Expected ')' but found ';'.
```

4. Reflection

In developing this mini compiler, several technical challenges faced us that deepened our understanding in compiler construction. The primary challenge was handling operator precedence properly. For that, the grammar had to be structured with separate production rules for each precedence level to make sure multiplication binds more tightly than addition. Another major challenge was the elimination of left recursion from grammar, as productions like `<expr> → <expr> '+' <term>` would introduce infinite recursion in recursive descent parsing. We solved it by redeveloping recursive productions as iterative loops, preserving the syntax of the language but making it suitable for top-down parsing. Designing informative error messages was also tricky; technical accuracy had to be balanced with the probable understanding of the user. The program reports on the error type and exact position. Token validation posed its own challenges about how to handle invalid characters and whether to continue parsing or to stop immediately. Similarly, managing dynamic memory allocation in C++ was a chore for a while, taking into consideration possible memory leaks in the structure of the parse tree-nodes containing pointers to children which need recursive deletion.

Given more time, several enhancements would make the compiler much more useful: error recovery mechanisms for the parser to allow it to resume parsing after encountering an error, so multiple problems could be detected in a single pass using techniques such as panic mode recovery by skipping tokens until a synchronizing point such as a semicolon is found. Semantic analysis would extend the compiler's function beyond syntax checking to provide meaningful analyses of program properties, such as type checking, variable declaration validation, and scope analysis. Enhancing the language to include statements of control, such as if-else statements and loops, would turn into a more practical compiler for authentic programming activities. Code generation would turn the compiler into a full compilation pipeline to generate executable output. And finally, migrating the C++ implementation to use Standard Template Library containers and smart pointers would considerably ease memory management issues and provide generally better code maintainability. In conclusion, this project demonstrated several key principles of compiler construction through hands-on implementation. It was particularly beneficial to experience some of the challenges during parsing, the design of grammar, and the strategy of error handling.

5.Reference

Dick Grune, Kees van Reeuwijk, Henri E. Bal, Criel J.H. Jacobs, Koen Langendoen, Modern Compiler Design, 2nd Edition, Springer, 2012.

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson Education.

Cooper, K. D., & Torczon, L. (2011). *Engineering a compiler* (2nd ed.). Morgan Kaufmann.

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.